Student Referral Sheet - Faculty of Computing (Final Examination Jan-June 2026)

Lec01 - Algorithm Basics

Algorithm = a finite, clear, step-by-step method for solving a problem. Input -> processing -> output. Required properties: correct, unambiguous, general, effective/simple and finite. Applications: sorting, searching, routing, data retrieval, shortest path and scheduling.

Algorithm Analysis

Purpose: predict time/memory before coding, compare alternatives, choose the efficient solution and understand growth when n becomes large. Cases: best case = minimum steps, worst case = maximum steps, average case = expected steps over inputs.

RAM Model - Exact Step Counting

Assume one processor and primitive operations take constant time. Count assignment, arithmetic, comparison, memory access, method/print call and return. In loops, count the final failed condition. Show each line count, total steps, then simplify only if the question asks Big O.

RAM Counting Templates

x = 100 -> 1 step. x = x + 100 -> assignment + addition = 2 steps. print x -> 1 step. while i <= 10: condition checked 11 times when body runs 10 times. for i = 1 to n: initialization once, comparison n+1, increment n, body n times.

Common Loop Patterns

One loop 1..n -> O(n). Two separate loops n+n -> O(n). Nested n by n -> O(n^2). Triangular loop 1+2+...+n = n(n+1)/2 -> O(n^2). i=i*2 or i=i/2 -> O(log n). n loop with inner log n -> O(n log n).

Asymptotic Notations

Big O = upper bound / at most growth, often worst-case. Omega = lower bound / at least growth, often best-case. Theta = tight bound / same upper and lower growth. Ignore constants and lower-order terms: 8n^2 + 4n + 20 -> O(n^2). Growth order: log n < n < n log n < n^2 < n^3 < 2^n.

Essay Style: Analysis Answer

Define the method first, then show why it is used. For runtime questions, write: input size n, basic operation, loop count/recurrence, total operations, final complexity and reason. Do not only write the final answer; show the path.

Structured Answer Keywords

"Trace" means show each iteration. "Illustrate" means draw/step through. "Analyze" means count or derive complexity. "Compare" means give differences in table form. "Implement" means write code/pseudocode using correct variables and boundary conditions.

Exact Step Count Example 1

for i=1 to n with body s=s+A[i]. Initialization = 1, comparison = n+1, increment = n. Body usually has memory access A[i], addition and assignment repeated n times. Final form can be written as an+b, therefore O(n).

Exact Step Count Example 2

Nested loop: for i=1 to n, for j=1 to i, print j. Inner body runs 1+2+...+n = n(n+1)/2 times. Therefore exact count has a quadratic term and complexity is O(n^2).

How to Simplify Big O

Drop constants: 4n -> n. Drop smaller terms: n^2+n -> n^2. For products keep all growing parts: n*log n -> n log n. For consecutive parts take dominant term. For branches use the branch that runs more in worst-case analysis.

Operation Count Checklist

For each line ask: how many times executed? Which primitive operations happen? Does it access array/list? Does the loop test run one extra time? Are there nested or dependent loops? Write a small table: statement, cost, times, total.

Lec02 - Insertion Sort

Insertion sort keeps the left part sorted. For each j, store key=A[j], compare with earlier elements, shift larger elements right, then insert key. Best case sorted array: O(n). Worst case reverse array: O(n^2). Space O(1). Stable and in-place.

Insertion Sort Trace Format

For each pass write: j value, key, comparisons, shifts, final array. Example wording: "key is compared with previous sorted items; all values greater than key shift one position right; key is placed at i+1." This is useful for structured questions.

Exact Insertion Sort Analysis

Use c_i for constant cost of each line and t_j for number of while-condition checks in pass j. Best case: while check once per pass -> linear. Worst case: while repeats many times -> arithmetic series -> quadratic.

Bubble + Selection Sort

Bubble sort: compare adjacent elements and swap if wrong order; largest moves to end each pass. With no-swap flag best O(n), otherwise usual best/worst O(n^2). Selection sort: repeatedly select minimum and swap to front; best/average/worst O(n^2), few swaps, in-place.

Stack Theory

Stack = LIFO. Push adds to top, pop removes top, peek reads top. Array stack: empty when top == -1; full when top == maxSize-1; count = top+1. Applications: recursion/function calls, undo, browser history, string reversal and expression evaluation.

Queue Theory

Queue = FIFO. Insert/enqueue at rear, remove/dequeue from front, peekFront reads first item. Empty when count == 0. A linear array queue can waste front spaces after removals. Use count to track items.

Circular Queue Theory

Circular queue wraps around the array. rear=(rear+1)%maxSize and front=(front+1)%maxSize. Full when count == maxSize; empty when count == 0. Display order starts from front and prints count items, not raw index 0..n.

Stack/Queue Exam Traps

Push usually stores at ++top; pop returns stack[top--]. Queue remove returns q[front] then moves front. Circular queue is not full when rear is at the last index; it is full only when count == maxSize.

Insertion Sort Essay Points

Important words: in-place, stable, adaptive, sorted prefix, key, shifting. Good answer: explain the sorted left subarray, then apply one pass. For reverse order, each key is compared with all previous items, causing the n(n-1)/2 pattern.

Queue Trace Method

Write front, rear, count after every operation. For circular queue also show actual array index after wrap. Logical order is not always index order; it starts from front and continues count steps.

Stack Trace Method

Draw stack vertically or show array indexes. After push, top increases. After pop, returned value is old top and top decreases. Empty stack has top=-1; do not access stackArr[-1].

Sorting Complexity Table

Insertion: best O(n), average/worst O(n^2), space O(1). Bubble: best O(n) with swapped flag, average/worst O(n^2). Selection: O(n^2) all cases. Merge: O(n log n) all cases, extra space O(n). Quick: average O(n log n), worst O(n^2). Heap: O(n log n), in-place.

Lec05 - Linked List

Linked list = nodes connected by links. Each node has data and next. Last node points to null. Advantages: dynamic size and easy insert/delete when position is known. Disadvantage: no direct index access; searching is sequential O(n).

Array vs Linked List

Array: fixed size, contiguous memory, direct index access O(1), insertion/deletion may need shifting. Linked list: dynamic, non-contiguous, sequential access O(n), insertion/deletion by relinking pointers.

Linear Search Returning Position

Use position counter. Start current=first and position=0. While current != null: if data/key matches return position; else current=current.next and position++. If end reached return -1. Mention it visits nodes one by one.

Lec06 - Tree Terminology

Tree = nodes connected by edges. Root = top node. Parent has children. Leaf has no children. Siblings share a parent. Path = sequence of connected nodes. Subtree = node with descendants. Degree = number of children. Depth = edges from root. Height = maximum depth.

Binary Search Tree - BST

BST rule: left child key < parent key, right child key >= parent key. Search/insert starts at root and compares. Smaller goes left; greater/equal goes right. Average O(log n) when balanced, worst O(n) when skewed.

Traversals

BFS/Level order = visit level by level. DFS: Inorder LNR = left, node, right; Preorder NLR = node, left, right; Postorder LRN = left, right, node. In a BST, inorder traversal prints keys in ascending order.

BST Delete Cases

Case 1 leaf: remove parent link. Case 2 one child: connect parent directly to child. Case 3 two children: replace node with inorder successor, then delete the successor from its old place. Successor = smallest node in right subtree: go right once, then keep going left.

Tree Drawing Rules

When inserting into BST, start at root each time. Write comparison beside the path: smaller -> left, greater/equal -> right. For traversal questions, first mark left subtree, root, right subtree. For deletion, identify case before changing links.

Complete vs Full Binary Tree

Full binary tree: every node has either 0 or 2 children. Complete binary tree: all levels are full except possibly last, and last level fills left to right. Heap must be complete. Height of complete tree with n nodes is floor(log2 n).

Data Structure Comparison

Stack and queue restrict access, so operations are O(1). Linked list grows dynamically but search is O(n). BST gives faster search only when height is small. Heap gives fast highest-priority removal but not full sorted search.

Linked List Operation Costs

insertFirst O(1). deleteFirst O(1). find/search O(n). insertAfter key O(n) because key must be found first. delete by key O(n). display O(n). The memory cost is one node object per item plus links.

Traversal Example Rule

For preorder, print root before subtrees. For inorder, print root between left and right. For postorder, print root after subtrees. If the tree is BST and question asks ascending order, use inorder.

Student ID :

Module Code :

Lec08 - Recursion

Recursion is a function calling itself to solve a smaller version of the same problem. It must have a base case to stop. Identify base case, recursive case, progress toward base case and returned value. Without a base case, recursion may become infinite.

Recurrence Equations

A recurrence expresses runtime using smaller input. Factorial style: T(n)=T(n-1)+O(1) -> O(n). Binary search: T(n)=T(n/2)+O(1) -> O(log n). Merge sort: T(n)=2T(n/2)+O(n) -> O(n log n).

Manual Recursion Computation

For g(n): if n%5==0 return n/5, else return g(n+1). g(3)->g(4)->g(5)->1. g(12)->g(13)->g(14)->g(15)->3. Show every recursive call until base case, then return back.

Master Method

Use for T(n)=aT(n/b)+f(n). Compare f(n) with n^(log_b a). If f(n) smaller -> case 1. If equal -> case 2. If larger and regularity holds -> case 3. Example T(n)=5T(n/5)+log n: n^(log_5 5)=n, log n is smaller, so T(n)=Theta(n).

Divide and Conquer

Three steps: divide into subproblems, conquer recursively, combine results. Quick sort does major work in partition/divide. Merge sort does major work in combine/merge. Balanced subproblems usually give n log n patterns.

Quick Sort + Merge Sort

Quick sort: partition around a pivot, then sort left and right partitions. Best/average balanced: Theta(n log n). Worst unbalanced: Theta(n^2). Merge sort: split into halves, recursively sort, merge sorted halves; best/average/worst O(n log n), extra memory for merge arrays.

Heap + Priority Queue

Heap is an array representing a complete binary tree. Max heap: parent >= children. Min heap: parent <= children. 1-based array: parent=floor(i/2), left=2i, right=2i+1. MAX_HEAPIFY O(log n), BUILD_MAX_HEAP O(n), HEAPSORT O(n log n), extract/insert O(log n).

Complexity Summary

Stack push/pop/peek O(1). Queue insert/remove/peek O(1). Linked list search O(n), insertFirst O(1). BST average search/insert/delete O(log n), worst O(n). Quick average O(n log n), worst O(n^2). Merge O(n log n). Heapify O(log n).

Master Theorem Mini Table

a=1,b=2,f=1 -> log n. a=2,b=2,f=n -> n log n. a=3,b=4,f=n^2 -> n^2 because f(n) is larger than n^(log_4 3). Always compute n^(log_b a) first, then compare with f(n).

Heap Trace Method

Index formulas are for 1-based arrays. To heapify, compare parent with left and right children; swap with largest child; continue at swapped child. In heapsort, sorted part grows at the right end of the array.

Final Quick Revision Order

Before exam: RAM steps, loop complexities, insertion trace, stack/queue conditions, linked-list pointer updates, BST traversal/delete, recursion base cases, Master theorem, quick/merge/heap complexities.

Student Referral Sheet - Faculty of Computing (Final Examination Jan-June 2026)

StackX - Array Stack

```
public class StackX {
    private int maxSize;
    private int[] stackArr;
    private int top;

    public StackX(int size) {
        maxSize = size;
        stackArr = new int[maxSize];
        top = -1;
    }
    public boolean isEmpty() { return top == -1; }
    public boolean isFull()  { return top == maxSize - 1; }
    public int getCount()    { return top + 1; }

    public void push(int value) {
        if (isFull())
            System.out.println("Stack is full");
        else
            stackArr[++top] = value;
    }
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack is empty");
            return -99;
        }
        return stackArr[top--];
    }
    public int peek() {
        if (isEmpty()) return -99;
        return stackArr[top];
    }
}
```

QueueArray - Linear Queue

```
public class QueueArray {
    private int maxSize, front, rear, count;
    private int[] queueArray;

    public QueueArray(int size) {
        maxSize = size;
        queueArray = new int[maxSize];
        front = 0; rear = -1; count = 0;
    }
    public boolean isEmpty() { return count == 0; }
    public boolean isFull()  { return rear == maxSize - 1; }
    public int getCount()    { return count; }

    public void insert(int item) {
        if (isFull()) {
            System.out.println("Queue is Full!");
            return;
        }
        queueArray[++rear] = item;
        count++;
    }
    public int remove() {
        if (isEmpty()) return -1;
        int temp = queueArray[front++];
        count--;
        return temp;
    }
    public int peekFront() {
        if (isEmpty()) return -1;
        return queueArray[front];
    }
}
```

CircularQueue - Wrap Around

```
public class CircularQueue {
    private int maxSize, front, rear, count;
    private int[] queueArray;

    public CircularQueue(int size) {
        maxSize = size;
        queueArray = new int[maxSize];
        front = 0; rear = -1; count = 0;
    }
    public boolean isEmpty() { return count == 0; }
    public boolean isFull()  { return count == maxSize; }

    public void insert(int item) {
        if (isFull()) return;
        rear = (rear + 1) % maxSize;
        queueArray[rear] = item;
        count++;
    }
    public int remove() {
        if (isEmpty()) return -1;
        int temp = queueArray[front];
        front = (front + 1) % maxSize;
        count--;
        return temp;
    }
    public void display() {
        int i = front;
        for (int c = 0; c < count; c++) {
            System.out.print(queueArray[i] + " ");
            i = (i + 1) % maxSize;
        }
    }
}
```

Singly Linked List - Core

```
class Link {
    public int id;
    public int marks;
    public Link next;

    public Link(int id, int marks) {
        this.id = id;
        this.marks = marks;
        this.next = null;
    }
    public void displayLink() {
        System.out.println(id + " " + marks);
    }
}

class LinkList {
    private Link first;

    public LinkList() { first = null; }
    public boolean isEmpty() { return first == null; }

    public void insertFirst(int id, int marks) {
        Link newLink = new Link(id, marks);
        newLink.next = first;
        first = newLink;
    }
    public Link find(int key) {
        Link current = first;
        while (current != null) {
            if (current.id == key) return current;
            current = current.next;
        }
        return null;
    }
    public boolean insertAfter(int key, int id, int marks) {
        Link current = first;
        while (current != null && current.id != key)
            current = current.next;
        if (current == null) return false;
        Link newLink = new Link(id, marks);
        newLink.next = current.next;
        current.next = newLink;
        return true;
    }
}
```

Linked List - Delete + Display

```
public Link deleteFirst() {
    if (first == null) return null;
    Link temp = first;
    first = first.next;
    return temp;
}

public boolean delete(int key) {
    if (first == null) return false;

    Link current = first;
    Link previous = null;

    while (current != null && current.id != key) {
        previous = current;
        current = current.next;
    }
    if (current == null) return false;

    if (current == first)
        first = first.next;
    else
        previous.next = current.next;
    return true;
}

public void displayList() {
    Link current = first;
    while (current != null) {
        current.displayLink();
        current = current.next;
    }
}
```

Structured Linked-List Samples

```
public int linearSearch(int target) {
    int position = 0;
    Link current = first;
    while (current != null) {
        if (current.id == target)
            return position;
        current = current.next;
        position++;
    }
    return -1;
}

public int countGreaterThan(int mark) {
    int count = 0;
    Link current = first;
    while (current != null) {
        if (current.marks > mark) count++;
        current = current.next;
    }
    return count;
}

public boolean updateMarks(int id, int newMarks) {
    Link current = find(id);
    if (current == null) return false;
    current.marks = newMarks;
    return true;
}
```

BST Node + Insert + Find

```
class Node {
    int empNo;
    String empName;
    Node leftChild, rightChild;

    public Node(int n, String name) {
        empNo = n;
        empName = name;
        leftChild = rightChild = null;
    }
    public void displayNode() {
        System.out.println(empNo + " " + empName);
    }
}

class Tree {
    private Node root;
    public Tree() { root = null; }

    public void insert(int n, String name) {
        Node newNode = new Node(n, name);
        if (root == null) { root = newNode; return; }

        Node current = root;
        Node parent;
        while (true) {
            parent = current;
            if (n < current.empNo) {
                current = current.leftChild;
                if (current == null) {
                    parent.leftChild = newNode; return;
                }
            } else {
                current = current.rightChild;
                if (current == null) {
                    parent.rightChild = newNode; return;
                }
            }
        }
    }
    public Node find(int key) {
        Node current = root;
        while (current != null && current.empNo != key) {
            if (key < current.empNo)
                current = current.leftChild;
            else
                current = current.rightChild;
        }
        return current;
    }
}
```

BST Recursive Find + Min/Max

```
public Node findRecursive(int key) {
    return findRecursive(root, key);
}
private Node findRecursive(Node current, int key) {
    if (current == null || current.empNo == key)
        return current;
    if (key < current.empNo)
        return findRecursive(current.leftChild, key);
    else
        return findRecursive(current.rightChild, key);
}

public Node minimum() {
    if (root == null) return null;
    Node current = root;
    while (current.leftChild != null)
        current = current.leftChild;
    return current;
}
public Node maximum() {
    if (root == null) return null;
    Node current = root;
    while (current.rightChild != null)
        current = current.rightChild;
    return current;
}
```

Tree Traversals

```
public void inOrder() { inOrder(root); }
private void inOrder(Node r) {       // L, N, R
    if (r != null) {
        inOrder(r.leftChild);
        r.displayNode();
        inOrder(r.rightChild);
    }
}
public void preOrder() { preOrder(root); }
private void preOrder(Node r) {      // N, L, R
    if (r != null) {
        r.displayNode();
        preOrder(r.leftChild);
        preOrder(r.rightChild);
    }
}
public void postOrder() { postOrder(root); }
private void postOrder(Node r) {     // L, R, N
    if (r != null) {
        postOrder(r.leftChild);
        postOrder(r.rightChild);
        r.displayNode();
    }
}
public void deleteAll() { root = null; }
public boolean isEmpty() { return root == null; }
// delete cases: leaf, one child, two children
// successor = right once, then left until null
```

Student ID :

Module Code :

Sorting Pseudocode

```
INSERTION_SORT(A)
for j = 1 to n-1
    key = A[j]
    i = j - 1
    while i >= 0 and A[i] > key
        A[i+1] = A[i]
        i = i - 1
    A[i+1] = key

BUBBLE_SORT(A)
for i = 0 to n-2
    swapped = false
    for j = 0 to n-i-2
        if A[j] > A[j+1]
            swap(A[j], A[j+1])
            swapped = true
    if swapped == false
        break

SELECTION_SORT(A)
for i = 0 to n-2
    min = i
    for j = i+1 to n-1
        if A[j] < A[min]
            min = j
    swap(A[i], A[min])
```

Quick Sort + Partition

```
PARTITION(A, p, r)
x = A[r]
i = p - 1
for j = p to r - 1
    if A[j] <= x
        i = i + 1
        exchange A[i] with A[j]
exchange A[i+1] with A[r]
return i + 1

QUICKSORT(A, p, r)
if p < r
    q = PARTITION(A, p, r)
    QUICKSORT(A, p, q - 1)
    QUICKSORT(A, q + 1, r)
```

Merge Sort + Merge Core

```
MERGESORT(A, p, r)
if p < r
    q = (p + r) / 2
    MERGESORT(A, p, q)
    MERGESORT(A, q + 1, r)
    MERGE(A, p, q, r)

MERGE(A, p, q, r)
n1 = q - p + 1
n2 = r - q
create L[0..n1-1], R[0..n2-1]
copy left and right halves
// compare first unmerged items
while i < n1 and j < n2
    if L[i] <= R[j]
        A[k] = L[i]; i++
    else
        A[k] = R[j]; j++
    k++
copy remaining L items
copy remaining R items
```

Heap Algorithms

```
MAX_HEAPIFY(A, i)
l = 2 * i
r = 2 * i + 1
if l <= A.heap_size and A[l] > A[i]
    largest = l
else
    largest = i
if r <= A.heap_size and A[r] > A[largest]
    largest = r
if largest != i
    exchange A[i] with A[largest]
    MAX_HEAPIFY(A, largest)

BUILD_MAX_HEAP(A)
A.heap_size = A.length
for i = floor(A.length / 2) downto 1
    MAX_HEAPIFY(A, i)

HEAPSORT(A)
BUILD_MAX_HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap_size = A.heap_size - 1
    MAX_HEAPIFY(A, 1)
```